

Continuous Integration, Delivery and Deployment

Nazih Errahel

erraheln@ex.istu.edu

Where are we now?

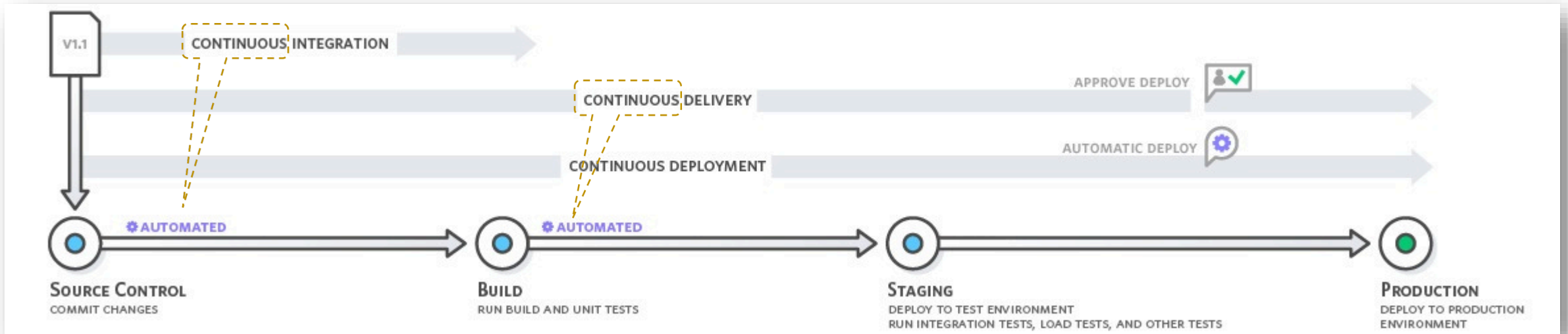
- 2. SDLC + WhyDevOps
- 3. DevOps Phases + Cloud Computing –Basics
- 4. VCS –GitLab
- 5. Containerization
- 6. Microservices
- 7. Automation – Ansible

8. CI/CD – Github Actions

OUTLINE

- What is Continuous Integration
- What is Continuous Delivery
- What is Continuous Deployment
- Github Actions CI/CD
- Github Actions Execution flow

Continuous X , $X \in \{Integration, Delivery, Deployment\}$

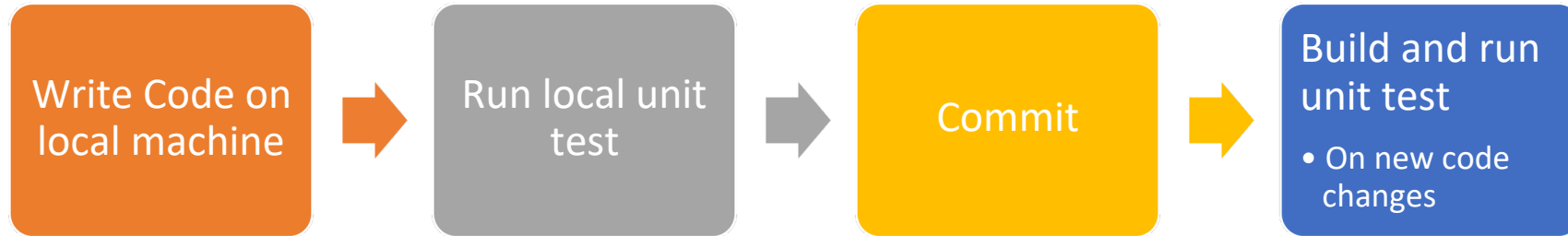


Continuous X , $X \in \{Integration, Delivery, Deployment\}$

$X = Integration$

- A development practice
- Integration of code changes **from multiple contributors**
- **Frequent commits**: Developers integrate their code changes early
 - often **to the main branch** of code repository.
- Every *commit**triggers **automated builds** and tests run
- **Key Goals**
 - to find and address **bugs** quicker, improve software **quality**
 - reduce the time it takes to **validate**--> **release** new software updates.

Continuous Integration workflow



How

- Maintain single source repo
- Automate the build
- Local testing
- Test in a production-like environment
- Make the latest executable available
- Make it transparent

Continuous Integration attributes

- Immediate change detection
 - Save time
- Code base **health**(code quality/code coverage)
- Automated **acceptance tests**
- Communication
 - If something fails, quickly notify the team
- Automated deployment process

Continuous Integration Benefits

- Better Developer Productivity
- Detect and Locate errors quickly
- Reduce risk of cost, schedule and budget
- No labor intensive manual testing by QA teams
- Deliver Updates Faster

“Continuous Integration doesn’t get rid of bugs, but it does make them dramatically easier to find and remove.”

-[Martin Fowler](#), Chief Scientist, Thought Works

Continuous X , $X \in \{Integration, Delivery, Deployment\}$

$X = Delivery$

Continuous Delivery

- Get the code ready for a release as quick as possible.
- Continuous integration practices Continuous delivery
- It has to have all the configuration necessary to push it into production.
 - Not just making sure your application passes basic automated unittest and integration tests Should pass E2E
 - testing, load testing, API reliability, performance testing, functional and non-function testing, localization testing, disaster recovery testing and many more through a continuous testing environment...
- Automatic TEST –BUILD
 - DEPLOY Manually

Continuous X , $X \in \{Integration, Delivery, Deployment\}$

$X = Deployment$

Continuous Deployment

- Automation of deployment process for pushing every successful build directly into **production** is generally known as Continuous Deployment.
- Refers to keeping your application **deployable** at any point.
- Frequent, automated deployment of the master branch to a production environment.
- Automatically (TEST-BUILD-DEPLOY)

CI/CD

- CI is the acronym for Continuous Integration
- CD can be the acronym for either Continuous Delivery or Continuous Deployment.
- Deployments no longer need to be an annual, quarterly, or a monthly event.
- With CI/CD
 - DevOps teams can deploy several times a day
- Automate build + test + deploy

Advantages of CI/CD

- **Fewer handoffs.**

- The more handoffs in your development pipeline, the more points of failure and the more complexity.
- *E.g. handoff mechanism in cellular network*

- **Increased development speed.**

- All stages of development are faster with CI/CD (due to automated processes).
- Make every team more efficient and developers can move onto other projects confidently.

- **More deployments.**

- One deployment / month ---> one deployment / few hours

- **Faster testing**

- Automated testing lets teams get feedback sooner and fail early rather than catching bugs in production – or worse – in the final release.

- **Fewer bugs.**

- With automated testing throughout the development process, bugs are caught as they arise
- Ensures better code quality overall and that every release works just as intended.

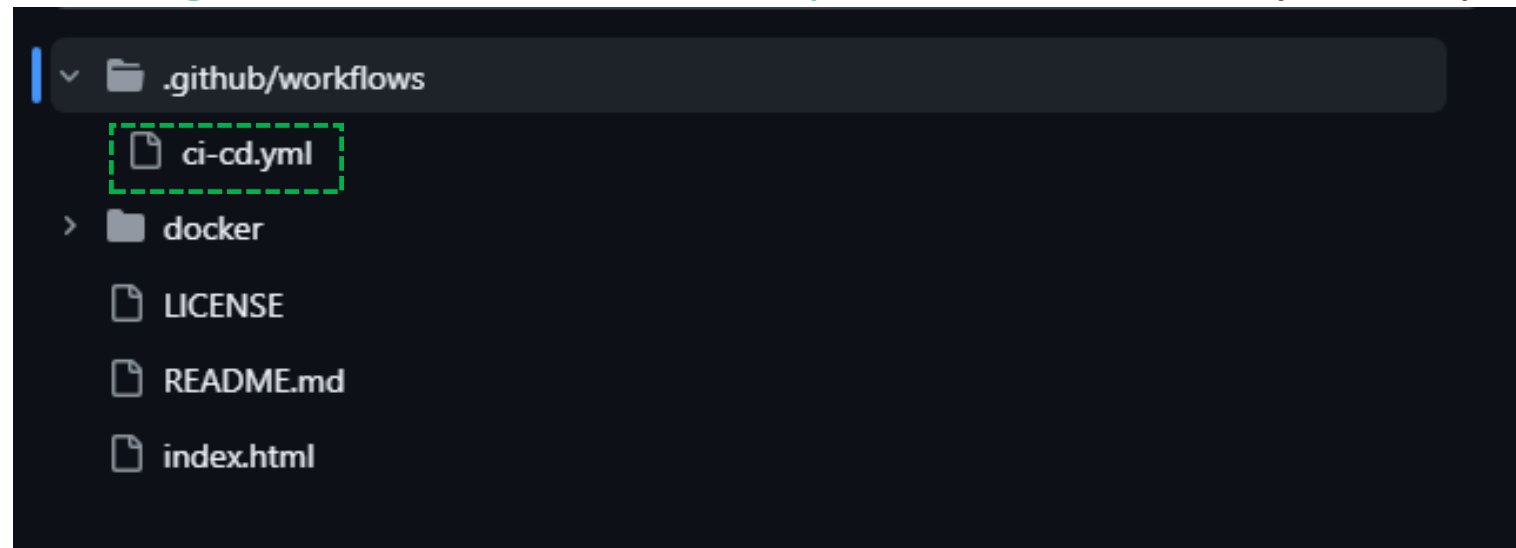
CICD Tools

- **Githubs Actions**
- GitLab
- CircleCI
- Jenkins

Where do you define the whole pipeline ?

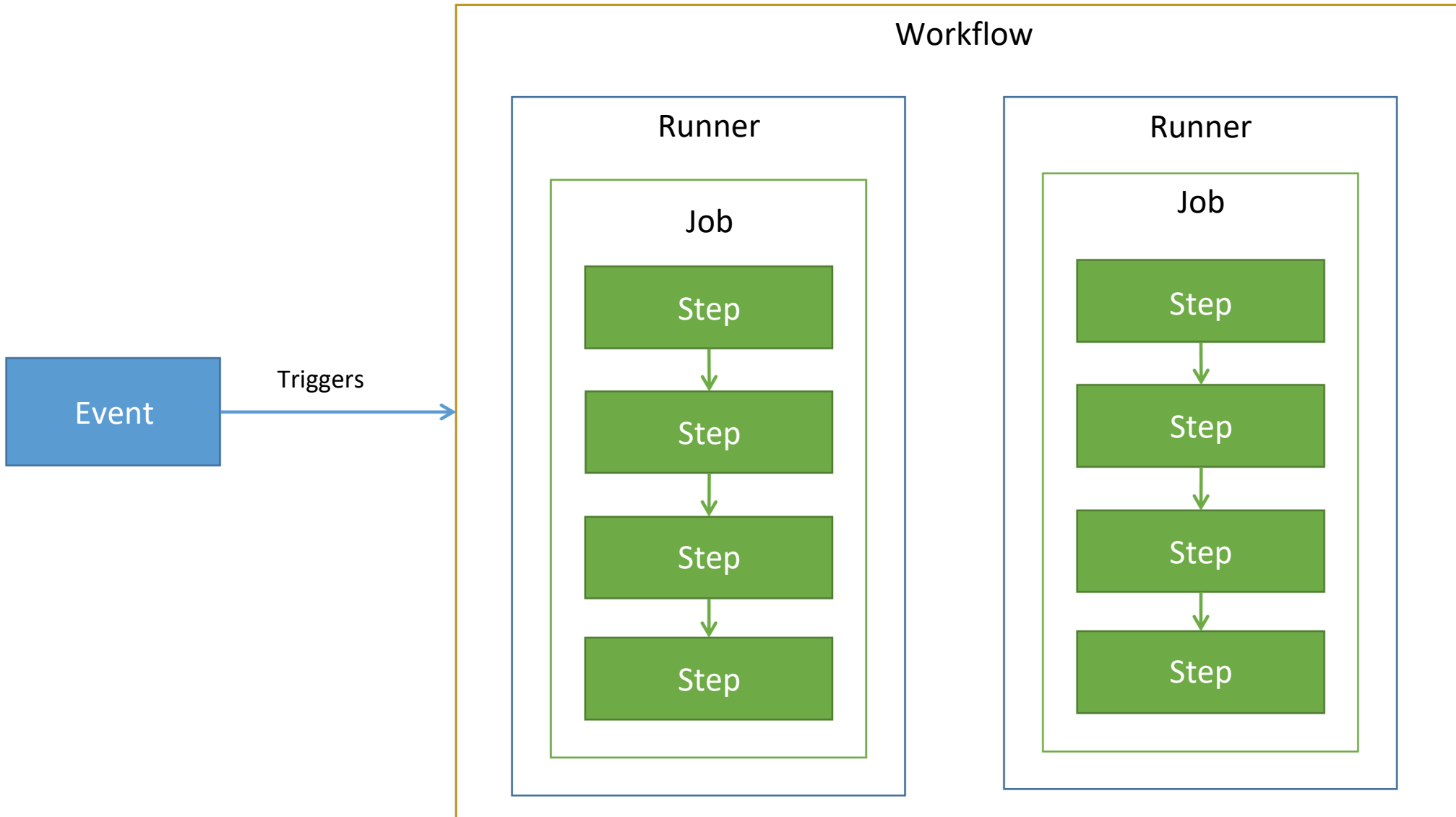
GitHub Actions Configuration File ([.github/workflows/ci-cd.yml](#))

- To use GitHub Actions, you need:
 - Code in Github repository
 - And the configuration file [.github/workflows/ci-cd.yml](#) in the root of your repository
 - Example



- [.github/workflows/ci-cd.yml](#) contains the CI/CD configuration
- This can be changed

GitHub Actions: Overview

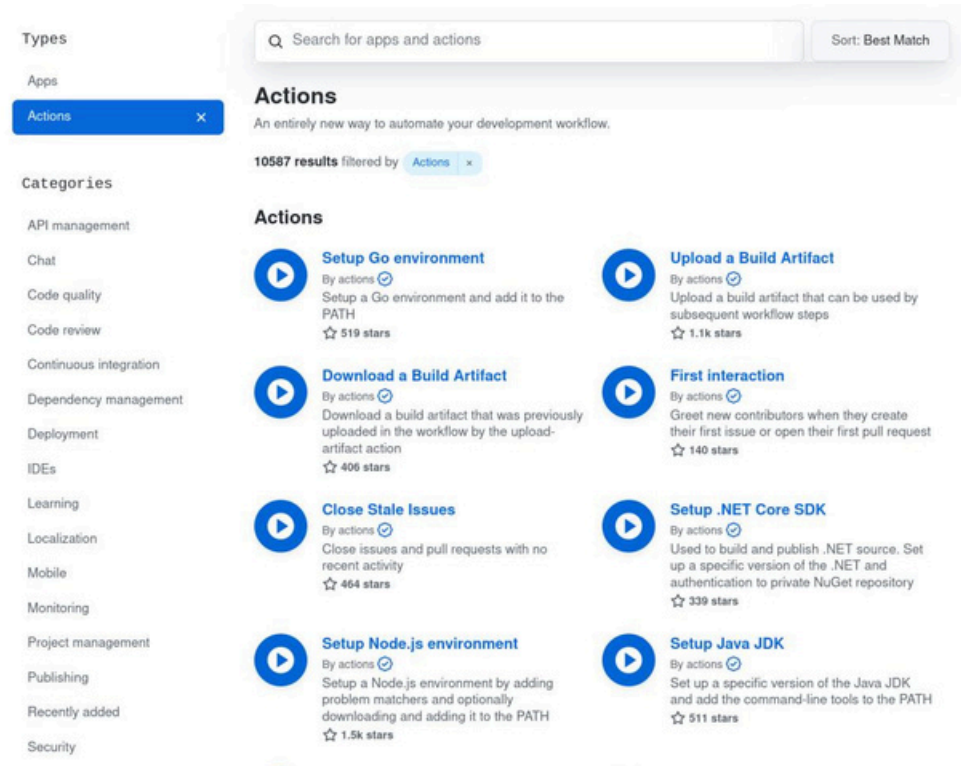


GitHub Actions: Action

A step can be just a sequence of commands or an Action

Actions are predefined and community-maintained scripts. Actions can receive inputs and produce outputs. You can write custom actions as Dockerfiles or using Javascript

GitHub marketplace offers a wide repository of community-maintained Actions



Some community Actions

Action	Description
JamesIves/github-pages-deploy-action@4.1.4	Deploy a folder to the gh-pages branch
pypa/gh-action-pypi-publish@master	Publish a package to PyPI
EnricoMi/publish-unit-test-result-action@v1	Generate python's unit test reports
actions/setup-python@v2	Setup a python environment
github/super-linter@v4	Run a combination of linters a generates a report
SamKirkland/FTP-Deploy-Action@4.1.0	Deploy a folder to an FTP server
elgohr/Publish-Docker-Github-Action@master	Deploy an image to the Docker Registry

GitHub Actions: Syntax

GitHub Actions uses YAML to define workflows

```
name: <workflow-name>
on: <events>

jobs:
  my-job-name:
    runs-on: <runner>
    steps:
      - name: <name-of-the-step>
        run: <command-to-execute>
      - name: <name-of-the-step>
        uses: <user>/<action>
        with:
          argument-1: <value-1>
          argument-2: <value-2>
```

Available runners hosted on GitHub:

- ubuntu-<latest | 20.04 | 18.04>
- macos-<latest | 11 | 10.5>
- windows-<latest | 2022 | 2019>

Events can be:

- push
- pull_request
- schedule
- ...

GitHub Actions: Example

```
name: Run unit tests
on: push

jobs:
  run-tests:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout the code
        uses: actions/checkout@main
      - name: Setup python 3.7
        uses: actions/setup-python@v1
        with:
          python-version: 3.7
      - name: Run the tests
        run: python -m unittest
```

GitHub Actions: Containers and Services

You can specify which container to use to run a job.

You can also specify additional containers needed to execute the job.

```
name: Run nodejs tests
on: push

jobs:
  run-tests:
    runs-on: ubuntu-latest
    container: node:lts-alpine
    services:
      redis:
        image: redis
    steps:
      - name: Checkout the code
        uses: actions/checkout@main
      - name: Install Dependencies
        run: npm install
      - name: Run tests that requires redis
        run: npm test
        env:
          REDIS_HOST: redis
```

GitHub Actions: Events

You can specify a list of events to capture. You can specify arguments to each event.

You can even schedule workflows to run using the cron syntax

```
on: [push, pull_request]
```

```
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
      - develop
```

```
on:
  schedule:
    - cron: "0 0 * * *"
```

GitHub Actions: Secrets

You can securely store secrets (passwords, tokens, etc...) in GitHub and they will be passed to the jobs. You can access the secrets using `${{ secrets.SECRET_NAME }}`

```
name: Publish to PyPI
on: push

jobs:
  build-n-publish:
    name: Build and publish to PyPI
    runs-on: ubuntu-18.04
    steps:
      - uses: actions/checkout@master
      - name: Set up Python 3.7
        uses: actions/setup-python@v1
        with:
          python-version: 3.7
      - name: Install
        run: python -m pip install build --user
      - name: Build binary wheel and a source tarball
        run: python -m build --sdist --wheel --outdir dist/ .
      - name: Publish to PyPI
        uses: pypa/gh-action-pypi-publish@master
        with:
          password: "${{ secrets.PYPI_API_TOKEN }}"
```


GitHub Actions: Complex Workflows

You can specify dependencies between jobs to create sequential and concurrent workflows

```
name: Run tests and then publish on PyPI
on: push

jobs:
  run-tests:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout the code
        uses: actions/checkout@main
      - name: Set up Python 3.7
        uses: actions/setup-python@v1
        with:
          python-version: 3.7
      - name: Run the tests
        run: python -m unittest
```

```
build-n-publish:
  name: Build and publish to PyPI
  runs-on: ubuntu-18.04
  needs: run-tests
  steps:
    - uses: actions/checkout@master
    - name: Set up Python 3.7
      uses: actions/setup-python@v1
      with:
        python-version: 3.7
    - name: Install
      run: python -m pip install build --user
    - name: Build binary wheel and a source tarball
      run: python -m build --sdist --wheel --outdir dist/ .
    - name: Publish to PyPI
      uses: pypa/gh-action-pypi-publish@master
      with:
        password: ${ secrets.PYPI_API_TOKEN }
```


GitHub Actions: Matrix

You can specify variables ranges and exploration strategies for the same job

```
jobs:
  run-browser-tests:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node: [6, 8, 10]
        browser: ["firefox", "chrome"]
    steps:
      - name: Setup node-${{ matrix.node }}
        uses: actions/setup-node@v2
        with:
          node-version: ${{ matrix.node }}
      - name: Install dependencies
        run: npm install
      - name: Run tests
        run: npm test
      env:
        BROWSER: ${{ matrix.browser }}
```

GitHub Actions: Code Reuse

You have to define a workflow that gets triggered on the event “workflow_call” and use the “uses” keyword in the callee job definition

```
# test.yaml
name: Reusable test
on: workflow_call

jobs:
  run-tests:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout the code
        uses: actions/checkout@main
      - name: Set up Python 3.7
        uses: actions/setup-python@v1
        with:
          python-version: 3.7
      - name: Run the tests
        run: python -m unittest
```

```
# push-pr.yaml
name: Run test on push or PR
on: [push, pull_request]

jobs:
  call-test:
    uses: <user>/<repo>/.github/workflows/test.yaml@main
```

References

- <https://hsf-training.github.io/hsf-training-cicd-github/01-introduction/index.html>
- <https://docs.gitlab.com/ee/ci/>
- <https://aws.amazon.com/devops/continuous-integration/>
- <https://www.cloudbees.com/continuous-delivery/continuous-integration>
- <https://www.atlassian.com/continuous-delivery/continuous-deployment>
- <https://github.com/aaquresh/CITraining>
- <https://www.atlassian.com/continuous-delivery/principles/continuous-integration-vs-delivery-vs-deployment>
- <https://about.gitlab.com/resources/ebook-single-app-cicd/>
- <https://www.slideshare.net/davidhahn1004/continuous-integrationdeployment-with-gitlab-ci>
- <https://www.slideshare.net/DanMAGIER/gitlab-cicd-122844742>
- <https://docs.gitlab.com/ee/ci/yaml/>
- <https://docs.gitlab.com/ee/ci/variables/index.html#custom-cicd-variables>
- https://docs.gitlab.com/ee/ci/yaml/gitlab_ci_yaml.html
- <https://jfreeman.dev/blog/2019/03/22/understanding-gitlab-runner/>
- <https://docs.gitlab.com/runner/>
- <https://docs.gitlab.com/runner/executors/>

Any Question ?

THANK YOU